GitHub Main Repo design
Repository Structure and Development Guidelines

1. Directory Structure

The repository is organized into separate modules, each (*or multiple*) maintained by a specific group. This structure allows teams to work independently while sharing a common codebase and configuration.

```
/
├── src/
│   ├── geometry/              # Managed exclusively by Group X
│   │   ├── ...
│   │   ├── README.md          # Module-specific documentation
│   │   ├── Cargo.toml         # Module-specific Cargo configuration (if needed)
│   │   └── lib.rs             # Module entry point
│   │
│   ├── types/                 # Managed exclusively by Group Y
│   │   └── ...
│   │
│   └── ...
│
├── Cargo.toml                 # Root Cargo configuration and dependencies
├── Cargo.lock
├── rustfmt.toml
├── CLAUDE.md
├── AGENTS.md
├── .gitignore
└── README.md
```

The root `Cargo.toml` defines the main project configuration and can import the individual modules as dependencies.

Each module HAS to contain its own `Cargo.toml`.

---

2. Permissions and Roles

Since the repository contains multiple components maintained by different teams, access control and code review responsibilities are managed through *GitHub Code Owners* and branch protection *rulesets*.

Each module is assigned to a responsible group, which acts as its Code Owner, and to the the repository manager which oversees the project and its integration process.

Pull requests modifying a module MUST receive approval from:

- At least one member of the group responsible for the affected module.
- The repository manager.

This is enforced by the 2.2 Rulesets.

This approach ensures that changes are reviewed both by the team responsible for the affected code and by the person overseeing the repository as a whole.

It also allows the rejection of contributions coming from other teams.

2.1 GitHub Teams

Groups are organized into GitHub Teams to simplify permission management and assign review responsibilities.

Each team is responsible for maintaining its assigned modules, reviewing contributions, ensuring that changes comply with the project's coding standards and managing issues regarding their module.

2.2 Rulesets

Branches, particularly `main`, are protected through GitHub Rulesets. These rules define:

- Which operations are allowed or prohibited on protected branches.
- Which users or roles can bypass the rules (*the Manager*).
- Which reviews are required before merging a pull request.
- Which automated checks must pass before a pull request can be merged into `main`.

The repository currently uses two rulesets.

Main branch ruleset

The `Basic rules for main` ruleset applies exclusively to the `main` branch and is actively enforced.

Its main requirements are:

- Branch protection: Branch deletion and non-fast-forward updates are prohibited.
- Pull request requirement: Changes must be submitted through a pull request.
- Review requirements: At least two approving reviews are required, including the required Code Owner review.
- Additional approval: An extra approval is required for changes that cannot be attributed to a Code Owner.
- Status checks: **All required automated checks MUST pass before merging**.
- Merge methods: Merge commits, squash merges, and rebase merges are permitted.

These settings are intended to ensure that changes entering `main` are reviewed and validated before integration.

> ***Full config :***


```
{
  "name": "Basic rules for main",
  "target": "branch",
  "enforcement": "active",
  "conditions": {
    "ref_name": {
      "exclude": [],
      "include": ["refs/heads/main"]
    }
  },
  "rules": [
    { "type": "deletion" },
    { "type": "non_fast_forward" },
    {
      "type": "pull_request",
```

```
      "parameters": {
        "required_approving_review_count": 2,
        "dismiss_stale_reviews_on_push": false,
        "required_reviewers": [],
        "require_code_owner_review": true,
        "dismissal_restriction": {
          "enabled": false,
          "allowed_actors": []
        },
        "require_last_push_approval": false,
        "required_review_thread_resolution": false,
        "require_extra_approval_for_unattributed_changes": true,
        "allowed_merge_methods": [
          "merge",
          "squash",
          "rebase"
        ]
      }
    },
    { "type": "update" },
    { "type": "creation" },
    {
      "type": "required_status_checks",
      "parameters": {
        "strict_required_status_checks_policy": false,
        "do_not_enforce_on_create": false,
        "required_status_checks": [
          { "context": "Cargo Check", "integration_id": 15368 },
          { "context": "Run Tests", "integration_id": 15368 },
          { "context": "Clippy Pedantic Lints", "integration_id": 15368 },
          { "context": "Check Code Formatting", "integration_id": 15368 }
        ]
      }
    }
  ]
}
```

Other branches ruleset

The `Basic rules for others` ruleset applies to all branches except `main` and is also actively enforced.

Its main requirements are:

- Branch deletion is prohibited.
- Non-fast-forward updates are prohibited.
- Authorized actors can bypass the ruleset.

Unlike the `main` ruleset, it does not require pull requests, approving reviews, or automated status checks.

This allows development branches to remain flexible while protecting them against deletion and history-rewriting operations.

> ***Full config :***

```
{
  "name": "Basic rules for others",
  "target": "branch",
  "enforcement": "active",
  "conditions": {
    "ref_name": {
      "exclude": ["refs/heads/main"],
      "include": ["~ALL"]
    }
  },
  "rules": [
    { "type": "deletion" },
    { "type": "non_fast_forward" }
  ]
}
```

---

3. Issue Management

Issues are created through *GitHub Issue Forms*, which provide a structured and consistent way to report bugs and request new features.

Using predefined forms makes issues easier to understand, categorize, assign, and track.

Issues are also automatically labeled and assigned through 6. Automated Workflows, reducing manual work and helping direct each issue to the appropriate team.

3.1 Issue Forms

GitHub Issue Forms are defined using YAML files stored in the repository's `.github/ISSUE_TEMPLATE/` directory.

Bug Report

The Bug Report form collects the information needed to investigate and reproduce a problem.

It includes:

- Group Name: Identifies the group reporting the issue.
- Feature: Specifies the affected component or module, such as Geometry or Types.
- What happened?: Describes the observed behavior and the expected outcome.
- Relevant error output: Collects logs, error messages, and error codes.
- Additional information: Provides space for any other useful context.
- Screenshots: Allows users to attach visual evidence when relevant.

The form automatically applies the `bug` label and uses the `[Bug]:` title prefix.

Feature Request

The Feature Request form collects requirements and proposals for extending existing functionality.

It includes:

- Group Name: Identifies the group submitting the request.

- Feature: Specifies the component or module that would be affected.
- What do you need?: Describes the requested functionality.
- Additional information: Provides context that may help with the design and implementation.
- Screenshots: Allows users to attach supporting visual material.

The form automatically applies the `feat` label and uses the `[Feat]:` title prefix.

Repository Changes Request

The Repository Changes Request form collects requirements and proposals for extending existing functionality.

It includes:

- Group Name: Identifies the group submitting the request.
- What do you need?: Describes the requested functionality.
- Screenshots: Allows users to attach supporting visual material.

The form automatically applies the `repo management` label and uses the `[Repo]:` title prefix.

This kind of issues get automatically assigned to the repository manager.

---

4. Testing

Tests are executed automatically through GitHub Actions.

Testing is performed alongside compilation checks, formatting validation, and Clippy lint checks.

5. Naming Conventions

Consistent naming conventions improve readability, simplify collaboration, and make the repository easier to maintain.

The following conventions are adopted for commits, pull requests, and branches.

5.1 Commits and Pull Requests

Commit messages and pull request titles must begin with one of the following prefixes:

- `feat:` — Introduces a new feature.
- `fix:` — Fixes a bug or incorrect behavior.
- `refactor:` — Restructures existing code without intentionally changing its behavior.
- `update:` — Updates existing functionality, configuration, or documentation.
- `other:` — Covers changes that do not fit into the preceding categories.

Examples:

```
feat: add vector normalization
fix: handle empty geometry input
refactor: simplify type conversion
```

```
update: improve module documentation
other: update repository metadata
```

5.2 Branches

Branch names must begin with one of the following prefixes:

- `feat-` — For new features.
- `fix-` — For bug fixes.
- `other-` — For all other changes.

Examples:

```
feat-vector-normalization
fix-empty-input
other-update-documentation
```

5.3 Convention Enforcement

These naming conventions **could** be checked automatically through GitHub Actions and enforced through additional repository rules where supported.

However, strict enforcement can introduce unnecessary complexity and cause avoidable failures over minor formatting mistakes.

For this reason, the recommended approach is to follow the conventions consistently and review names carefully during development and code review. Automated enforcement can be introduced later if it becomes necessary.

6. Automated Workflows

GitHub Actions automates routine development and quality-assurance tasks, helping maintain consistency across the repository.

The current workflows are intended to:

- Label and assign issues: Automatically apply the appropriate labels and assign issues to the relevant group members.
- Check compilation: Run `cargo check` to detect compilation errors.
- Run tests: Execute `cargo test` to verify the correctness of the code.
- Check formatting: Run `rustfmt` validation to ensure consistent code formatting.
- Run Clippy: Apply pedantic lint checks to identify potential bugs, questionable patterns, and code-quality issues.

These workflows run automatically according to their configured triggers. For pull requests targeting `main`, all four required status checks must pass before merging:

1. `Cargo Check`
2. `Run Tests`
3. `Clippy Pedantic Lints`
4. `Check Code Formatting`

If any required check fails, the pull request cannot be merged until the issue is resolved or an authorized bypass is used.